Modeling and Control of A Planar Quadrotor

Konstantinos Spinos

***Abstract*—Modern Predictive Control (MPC) is an optimal control technique that uses a model of a system to predict it's behavior and optimize control actions over a finite horizon. It is widely used in robotics since it handles well non linear MIMO systems with constraints. In this paper, systematic analysis and control will be given first using Linear Quadratic Regulator (LQR) and then using MPC as a Quadratic Programming (QP) problem with respect to the inherent constraints of the system.**

I. Introduction

The paper concerns the modeling and control of non-linear systems using the Linear Quadratic Regulator (LQR) and Convex Model Predictive Control (MPC). In particular, we are going to model, simulate and control a planar quadrotor with aerodynamic drag (Fig. 1).

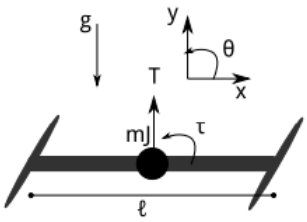


Fig. 1. Planar Quadrotor System

II. Modeling

The system is a rigid body where the propellers exert some force T and some torque τ on it (see Fig. 1). T and τ are the control inputs. The system has six continuous state variables: position x, y, orientation θ, and their time derivatives $\dot{x}, \dot{y}, \dot{\theta}$. We are also modeling aerodynamic drag as damping in the local velocity frame. The dynamics of the system along with the state and the control input is as follows:

$$f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \frac{1}{m}(-sin\theta(T + f_y) + cos\theta f_x) \\ \frac{1}{m}(cos\theta(T + f_y) + sin\theta f_x) - g \\ \frac{\tau - f_\theta}{J} \end{bmatrix} \in \mathbb{R}^6$$

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \\ \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} \in \mathbb{R}^6 \quad \mathbf{u} = \begin{bmatrix} T \\ \tau \end{bmatrix} \in \mathbb{R}^2 \quad , \; given \; that:$$

- $f_x = -k_x v_x |v_x|, \quad k_x = 0.3$
- $f_y = -k_y v_y |v_y|, \quad k_y = 0.3$
- $f_\theta = -k_\theta \dot{\theta}, \quad k_\theta = 0.02$

The mass of the system is $m = 1kg$ and the acceleration of the quadrotor due to gravity is $g = 9.81 \; ms^2$. The terms v_x, v_y represent the local frame velocities of the rigid body. We introduce the rotation matrix R:

$$R = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix}$$

The velocities v_x, v_y can be correlated with the word frame velocity $\dot{x}$ of the quadrotor through the matrix R, and we can immediately compute them as:

$$v = \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} \dot{x}cos\theta + \dot{y}sin\theta \\ -\dot{x}sin\theta + \dot{y}cos\theta \end{bmatrix}$$

A simple visualization of the system can be seen in the image below for $(x, y) = (0.8, 0.9)$ and $u = \begin{bmatrix} mg & 0 \end{bmatrix}^T$ (hover point). For the plot figure, maptplot library has been used.

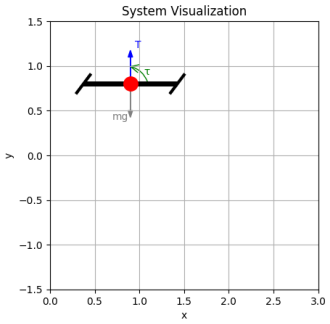
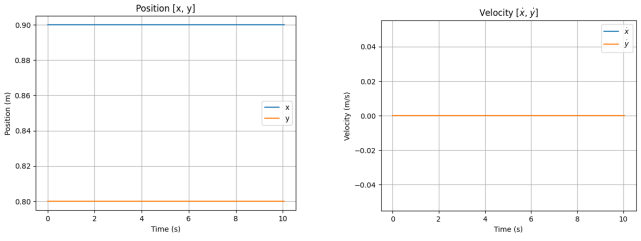


Fig. 2. System Visualization



(a) Position (b) Velocity

Fig. 3. Position and velocity tracking

For this, we discretized the system using Runge-Kutta 4th Order (RK4). The integration steps are as follows:

$$
\begin{aligned}
f_1 &= f(\mathbf{x}, \mathbf{u}) \\
f_2 &= f(\mathbf{x} + f_1 \frac{dt}{2}, \mathbf{u}) \\
f_3 &= f(\mathbf{x} + f_2 \frac{dt}{2}, \mathbf{u}) \\
f_4 &= f(\mathbf{x} + f_3 dt, \mathbf{u}) \\
x_{next} &= x + \frac{dt}{6}(f_1 + 2f_2 + 2f_3 + f_4)
\end{aligned}
$$

III. Linearization and LQR

In this section, we shall control the system to track the hover point using Linear Quadratic Regulator (LQR). Before diving into LQR, the system should be linearized around the hover point $\bar{\mathbf{x}}, \bar{\mathbf{u}}$. Consequently, by performing Taylor Series around $\bar{\mathbf{x}}, \bar{\mathbf{u}}$:

$$
\boldsymbol{x}_{k+1} = f_d(\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}}) + \left.\frac{\partial f_{\mathrm{d}}}{\partial \boldsymbol{x}}\right|_{\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}}} (\boldsymbol{x}_k - \bar{\boldsymbol{x}}) + \left.\frac{\partial f_{\mathrm{d}}}{\partial \boldsymbol{u}}\right|_{\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}}} (\boldsymbol{u}_k - \bar{\boldsymbol{u}})
$$

where f_d stands for the discretized dynamics function of the system through Runke-Kutta 4th Order. We are only interested in the partial derivatives of f_d for the LQR, and later for the MPC control.

$$
\boldsymbol{A} = \left.\frac{\partial f_{\mathrm{d}}}{\partial \boldsymbol{x}}\right|_{\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}}} \qquad \boldsymbol{B} = \left.\frac{\partial f_{\mathrm{d}}}{\partial \boldsymbol{u}}\right|_{\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}}}
$$

We can finally perform infinite LQR, which returns the matrices P_{inf}, K_{inf} to build the controller:

$$
\boldsymbol{u} - \bar{\boldsymbol{u}} = -K_{inf}(\boldsymbol{x} - \bar{\boldsymbol{x}}) \Rightarrow \Delta \boldsymbol{u} = -K_{inf} \Delta \boldsymbol{x}
$$

The general form of LQR for our system is as follows.

$$
\begin{aligned}
&J = \textstyle\sum_{k=1}^{K-1} (\frac{1}{2} \Delta \boldsymbol{x}_k^T \boldsymbol{Q}_k \Delta \boldsymbol{x}_k + \frac{1}{2} \Delta \boldsymbol{u}_k^T \boldsymbol{R}_k \Delta \boldsymbol{u}_k) + \frac{1}{2} \Delta \boldsymbol{x}_K^T \boldsymbol{Q}_K \Delta \boldsymbol{x}_K \\
&s.t.\ \Delta \boldsymbol{x}_{k+1} = \boldsymbol{A}_k \Delta \boldsymbol{x}_k + \boldsymbol{B}_k \Delta \boldsymbol{u}_k \\
&\boldsymbol{Q} \geq 0 \\
&\boldsymbol{R} \geq 0
\end{aligned}
$$

To this end, we perform Riccati recursion long enough to ensure convergence choosing K around 100-200. The recursion equations are as follows:

$$
\begin{aligned}
\boldsymbol{K}_k &= (\boldsymbol{R}_k + \boldsymbol{B}_k^T \boldsymbol{P}_{k+1} \boldsymbol{B}_k)^{-1} \boldsymbol{B}_k^T \boldsymbol{P}_{k+1} \boldsymbol{A}_k \\
\boldsymbol{P}_k &= \boldsymbol{Q}_k + \boldsymbol{A}_k^T \boldsymbol{P}_{k+1} (\boldsymbol{A}_k - \boldsymbol{B}_k \boldsymbol{K}_k)
\end{aligned}
$$

At the end of the recursion, we set $P = Q_N$, where Q_N refers to the terminal cost, and take the first P_k, K_k. For this case, the best overall set of the matrices $\boldsymbol{Q}, \boldsymbol{R}$ are found to be:

$$
\boldsymbol{Q} = \begin{bmatrix} 1 & 1 & 80 & 1 & 1 & 80 \end{bmatrix}^T \qquad \boldsymbol{R} = \begin{bmatrix} 0.1 & 0.1 \end{bmatrix}^T
$$

$$
\boldsymbol{Q} = \begin{bmatrix} 1 & 1 & 80 & 5 & 1 & 80 \end{bmatrix}^T \qquad \boldsymbol{R} = \begin{bmatrix} 0.1 & 0.1 \end{bmatrix}^T
$$

This implies that (x, y) do not need a significant correction to track their desired state, although θ is heavily "punished", which is expected since the system is inherently sensitive to angular movements along the y axis. Throughout multiple simulations, it can be found that the control limit of LQR settles approximately around 70 meters, at any angle. The further we set the initial point of the simulation, the slower the convergence, leading to a simulation time of 30 seconds. The results for various starting points can be seen in the following.

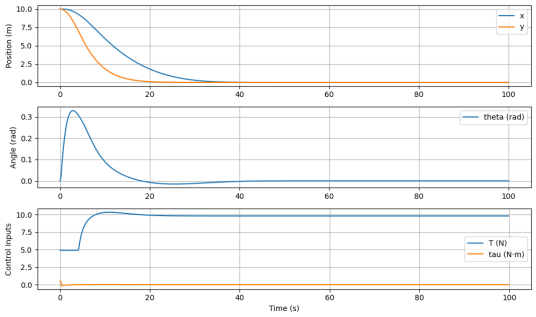


Fig. 4. $x_0 = [10,\ 10,\ 0,\ 0,\ 0,\ 0]$

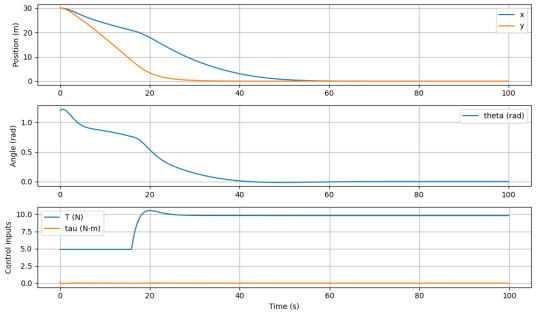


Fig. 5. $x_0 = [30,\ 30,\ 1.2,\ 0,\ 0,\ 0]$

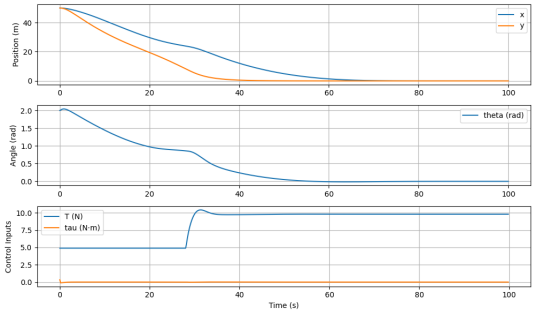


Fig. 6. $x_0 = [50,\ 50,\ 2.0,\ 0,\ 0,\ 0]$

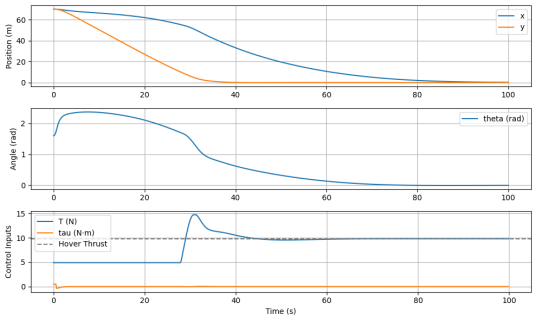


Fig. 7. $x_0 = [70,\ 70,\ 1.6,\ 0,\ 0,\ 0]$

In the simulations in the above, the reference state and the linearization point were chosen to be the same. That is not necessary, as long as we linearize along a fixed point. For this scenario, we choose as the reference point: $\boldsymbol{x}_{ref} = \begin{bmatrix}10 & 10 & 0 & 0 & 0 & 0\end{bmatrix}$ and as the linearization point the initial state: $\boldsymbol{x}_{fixed} = \begin{bmatrix}0 & 0 & 0 & 0 & 0 & 0\end{bmatrix}$

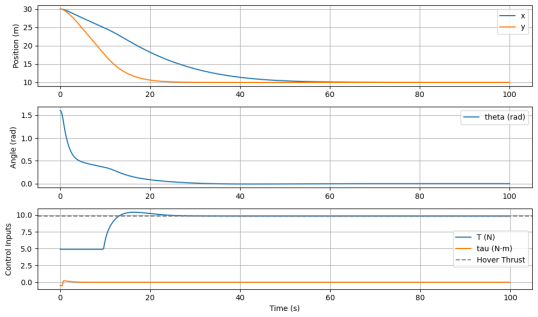


Fig. 8. $x_0 = [30,\ 30,\ 1.6,\ 0,\ 0,\ 0]$

LQR works very well, but when introducing input-control constraints, it blows up from afar. In the next section, we introduce the Modern Predictive Control (MPC) for a more robust approach.

IV. Controlling via Convex MPC

The general form of the MPC's minization function is as follows:

$$\sum_{k=1}^{H-1}\left(\tfrac{1}{2}\Delta\boldsymbol{x}_k^T\boldsymbol{Q}_k\Delta\boldsymbol{x}_k + \tfrac{1}{2}\Delta\boldsymbol{u}_k^T\boldsymbol{R}_k\Delta\boldsymbol{u}_k\right) + \tfrac{1}{2}\Delta\boldsymbol{x}_H^T\boldsymbol{P}_H\Delta\boldsymbol{x}_H$$

$$s.t. \Delta\boldsymbol{x}_{k+1} = \boldsymbol{A}_k\Delta\boldsymbol{x}_k + \boldsymbol{B}_k\Delta\boldsymbol{u}_k + c$$

The difference here is that we choose the horizon of the simulation and the steps taken inside of it. In addition, the value function contains the $\boldsymbol{P}_H$ matrix from the infinite LQR implementation to ensure stability and convergence at a longer distance. The MPC without this term solves for a finite horizon of our choice. By adding the matrix $\boldsymbol{P}_H$, we achieve convergence for very long distances as the recursion continues to run for infinity. By solving MPC as Quadratic Programming (QP), the cost function transforms to

$$J = \begin{bmatrix}\boldsymbol{z}^T\boldsymbol{H}\boldsymbol{z} + 2\boldsymbol{h}^T\boldsymbol{z}\end{bmatrix}$$
$$s.t.\ \boldsymbol{G}\boldsymbol{z} - \boldsymbol{d} = \boldsymbol{0}$$

$$\boldsymbol{z} = \begin{bmatrix}\boldsymbol{u}\\ \boldsymbol{x}\\ \boldsymbol{u}\\ \vdots\\ \boldsymbol{x}\end{bmatrix} \quad \boldsymbol{h} = -\begin{bmatrix}\boldsymbol{R}\boldsymbol{u}_r\\ \boldsymbol{Q}\boldsymbol{x}_r\\ \boldsymbol{R}\boldsymbol{x}_r\\ \vdots\\ \boldsymbol{P}_H\boldsymbol{x}_r\end{bmatrix} \quad \boldsymbol{H} = \begin{bmatrix}\boldsymbol{R} & & & & \\ & \boldsymbol{Q} & & & \\ & & \boldsymbol{R} & & \\ & & & \ddots & \\ & & & & \boldsymbol{P}_H\end{bmatrix}$$

$$\boldsymbol{G} = \begin{bmatrix}-\boldsymbol{B} & \boldsymbol{I} & \boldsymbol{0} & \dots & \\ \boldsymbol{0} & -\boldsymbol{A} & -\boldsymbol{B} & \boldsymbol{I} & \dots \\ \vdots & \ddots & \ddots & \ddots & \\ \boldsymbol{0} & \dots & -\boldsymbol{A} & -\boldsymbol{B} & \boldsymbol{I}\end{bmatrix} \quad \boldsymbol{d} = \begin{bmatrix}\boldsymbol{A}\boldsymbol{x} + c\\ c\\ c\\ \vdots\\ c\end{bmatrix}$$

x_r represents the reference point. The dimensions of each vector/matrix are as follows.

$$\boldsymbol{z} \in \mathbb{R}^{K(N+M)\times 1}$$
$$\boldsymbol{d} \in \mathbb{R}^{K(N+M)\times 1}$$
$$\boldsymbol{H} \in \mathbb{R}^{K(N+M)\times K(N+M)}$$
$$\boldsymbol{G} \in \mathbb{R}^{(K\cdot N)\times K(N+M)}$$
$$\boldsymbol{d} \in \mathbb{R}^{(K\cdot N)\times 1}$$

Since the problem will be solved by the QP solver proxsuite, we shall also define the matrix $\boldsymbol{C}$ together with the inequality constraints on the control input $\boldsymbol{lb}, \boldsymbol{ub}$.

$$\boldsymbol{lb} = \begin{bmatrix}\boldsymbol{u}_{min}\\ \boldsymbol{u}_{min}\\ \boldsymbol{u}_{min}\\ \vdots\\ \boldsymbol{u}_{min}\end{bmatrix} \quad \boldsymbol{ub} = \begin{bmatrix}\boldsymbol{u}_{max}\\ \boldsymbol{u}_{max}\\ \boldsymbol{u}_{max}\\ \vdots\\ \boldsymbol{u}_{max}\end{bmatrix} \quad \boldsymbol{C} = \begin{bmatrix}\boldsymbol{I} & \boldsymbol{0} & \boldsymbol{0} & \dots & \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} & \dots \\ \vdots & \ddots & \ddots & \ddots & \\ \boldsymbol{0} & \dots & & \boldsymbol{I} & \boldsymbol{0}\end{bmatrix}$$

where $\boldsymbol{u}_{min} = [0.5mg,\ -0.5]$ and $\boldsymbol{u}_{max} = [1.5mg,\ 0.5]$. The matrix $\boldsymbol{C}$ extracts all the control inputs on the horizon K and $\boldsymbol{lb}, \boldsymbol{ub}$ force the physical limits. The MPC works much better than the LQR, achieving convergence from 200+ meters afar. To manage such distances, the weight over θ should be very large. The optimal weight matrices $\boldsymbol{Q}, \boldsymbol{R}$ have been found to be as follows.

$$\boldsymbol{Q} = \begin{bmatrix}1 & 1 & 500 & 1 & 1 & 500\end{bmatrix}^T \quad \boldsymbol{R} = \begin{bmatrix}0.1 & 0.1\end{bmatrix}^T$$

$$\boldsymbol{Q} = \begin{bmatrix}1 & 1 & 2000 & 10 & 100 & 2000\end{bmatrix}^T \quad \boldsymbol{R} = \begin{bmatrix}0.1 & 0.1\end{bmatrix}^T$$

The controller can follow the desired state from more than 200 meters, but we set the limit here since the simulation time reaches approximately 60 seconds. For such big weights, it is expected that the quadrotor slowly tracks the desired position. In the following, one can see some simulations from various starting points. In practice, such weights may not be feasible, thus reducing the maximum distance that ensures convergence.

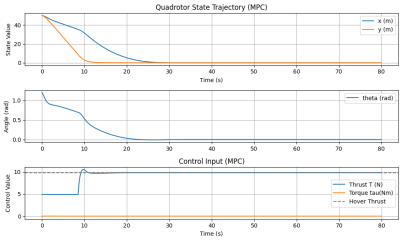


Fig. 9. $x_0 = [50,\ 50,\ 1.2,\ 0,\ 0,\ 0]$

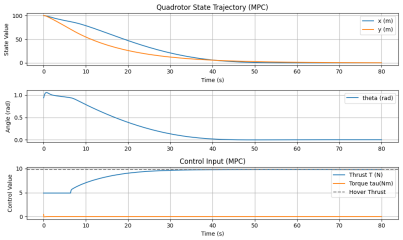


Fig. 10. $x_0 = [100,\ 100,\ 0.94,\ 0,\ 0,\ 0]$

For the figures in the above, the horizon is set to 10, which implies that for a simulation timestep $dt = 0.05$, we look and update every 0.5 seconds ahead. Setting the horizon higher decelerates the simulations, but could provide better results. Setting it lower gives fast convergence, but for shorter distances. By testing different values for the weights of the matrices $\boldsymbol{Q}$ and $\boldsymbol{R}$, we can get satisfactory results for any horizon. Above, we can witness the extreme example of reaching the hover state from 200 meters afar.

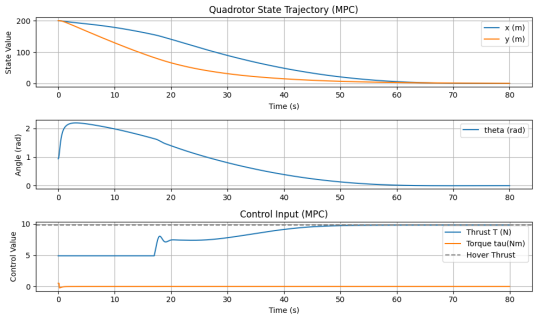


Fig. 11. $x_0 = [200,\ 200,\ 0.94,\ 0,\ 0,\ 0]$

Modern Predictive Control can converge to any point, not just the fixed point [0, 0, 0]. In the figure above, the reference point is $[30,\ 20,\ 0,\ 0,\ 0,\ 0]$, while the linearization takes place in the fixed point. The angle θ is zero for convenience reasons, but any value is acceptable as long as the control input $\boldsymbol{u}$ is set accordingly.

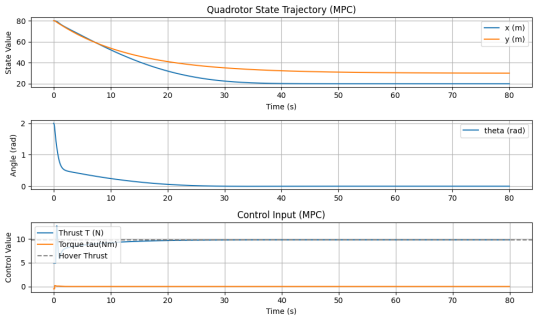


Fig. 12. $x_0 = [80,\ 80,\ 2.0,\ 0,\ 0,\ 0]$

We introduce Gaussian noise with zero mean in the system. Specifically, the covariance matrix is chosen as follows:

$$\boldsymbol{\Sigma} = \begin{bmatrix} 0.01 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.01 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.0025 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.0025 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.0025 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.0025 \end{bmatrix}$$

Testing against the previous example, we notice that noise strongly affects the control inputs and θ is the most sensitive to it. By increasing the values in the covariance matrix, the noise effect becomes stronger and can fully cover the true signal.

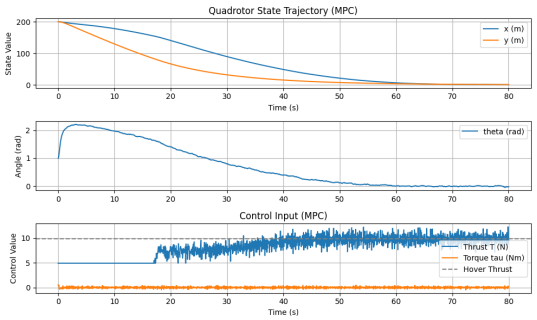


Fig. 13. MPC with noise

Finally, to test in a real-word environment, the MPC controller will be used for the quadrotor to follow a 2D path/trajectory. For that, we choose a random circular trajectory with a radius of 20 meters. With the current weights applied by the matrices $\boldsymbol{Q}$ and $\boldsymbol{R}$, the angle θ is heavily penalized while the positions x, y are not, leading to poor tracking of the trajectory. For the simulation below, the $\boldsymbol{Q}, \boldsymbol{R}$ matrices are as follows.

$$\boldsymbol{Q} = \begin{bmatrix} 100 & 100 & 10 & 100 & 100 & 10 \end{bmatrix}^T \quad \boldsymbol{R} = \begin{bmatrix} 0.1 & 0.1 \end{bmatrix}^T$$

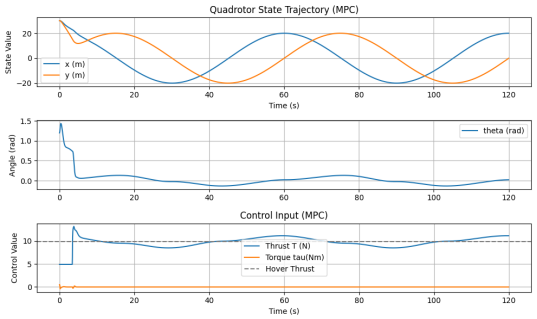


Fig. 14. x, y, θ of the quadrotor

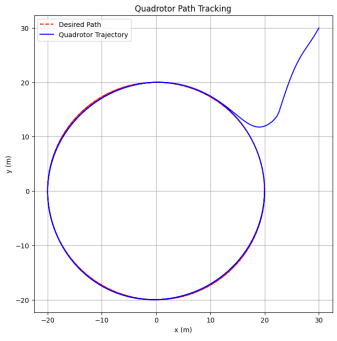


Fig. 15. Trajectory tracking

As can be witnessed, by penalizing the positions, the quadrotor stays in course and by reducing the weights in the angle of the quadrotor, it is more flexible to move accordingly to the desired state.